

3. 신경망

퍼셉트론으로 복잡한 함수를 표현할 수 있게 되었지만, 가중치를 설정하는 작업은 수동으로

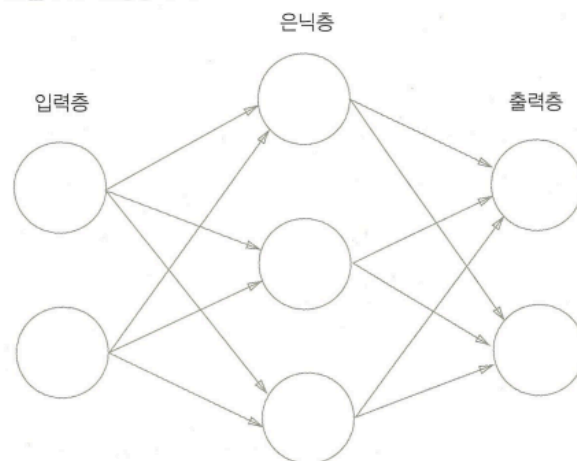
3.1. 퍼셉트론 → 신경망

3.1.1 신경망

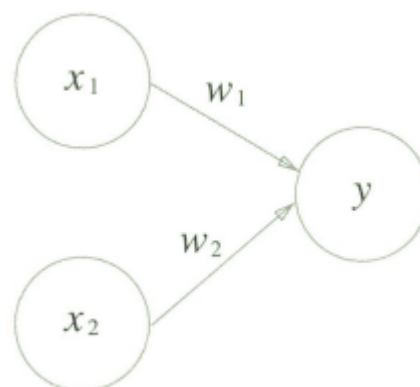
입력층은 0층이며 입력층부터 차례로 1, 2층이라 지정

아래의 그림에선 출력층은 2층이며, 3층으로 구성되어 있다.

그림 3-1 신경망의 예



3.1.2 퍼셉트론 복습

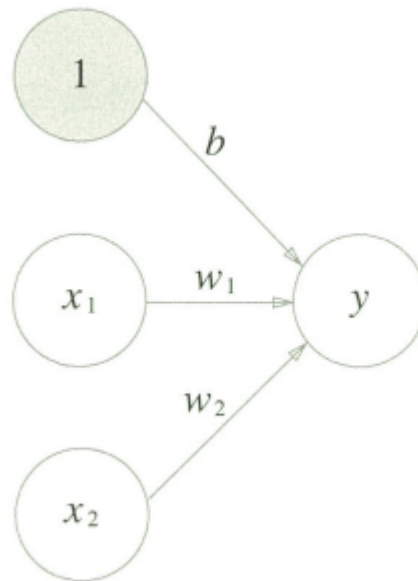


이전시간 복습

$$y = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$

x 신호를 입력받아 y 출력, b 편향, w 가중치

그림 3-3 편향을 명시한 퍼셉트론



위의 수식 퍼셉트론화

$$y = h(b + w_1x_1 + w_2x_2)$$

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases}$$

더 간단히 식으로 나타내면..

3.1.3 활성화 함수

: 위의 $h(x)$, 즉 입력신호들을 출력 신호로 변환하는 함수

⇒ 입력 신호의 총합이 활성화를 일으키는지 정하는 역할

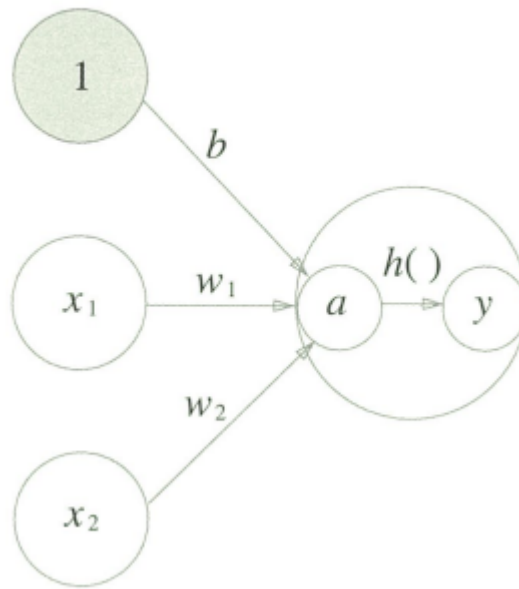
$$a = b + w_1x_1 + w_2x_2$$

[식 3.4]

$$y = h(a)$$

[식 3.5]

그림 3-4 활성화 함수의 처리 과정



가중치 신호까지 반영하여 조합 → a 노드 —활성화 함수h()→ h(a) → y 노드(=y 뉴런)

3.2 활성화 함수

활성화 함수는 임계값을 경계로 출력이 바뀐다.

▶ 활성화 함수는 "계단 함수" 이다.

3.2.1 시그모이드 함수

: 신경망에서 자주 이용하는 활성화 함수중 하나

$$h(x) = \frac{1}{1 + \exp(-x)}$$

[식 3.6]

exp= e^{-x}

3.2.2 계단 함수 구현

```
>>> def step_function(x):
...     y = x>0
...     return y.astype(np.int)
...
>>> import numpy as np
>>> x = np.array([-1.0, 1.0, 2.0])
>>> x
array([-1.,  1.,  2.])
>>> y = x>0
>>> y
array([False,  True,  True])
```

```
>>> y = y.astype(np.int)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "C:\Users\rkdus3696\anaconda3\Lib\site-packages\numpy\__init__.py", line 324, in __getattr__
    raise AttributeError(__former_attrs__[attr])
AttributeError: module 'numpy' has no attribute 'int'.
`np.int` was a deprecated alias for the builtin `int`. To avoid this error in existing code, use `int` by itself. Doing
this will not modify any behavior and is safe. When replacing `np.int`, you may wish to use e.g. `np.int64` or `np.int32`
to specify the precision. If you wish to review your current use, check the release note link for additional informati
on.
The aliases was originally deprecated in NumPy 1.20; for more details and guidance see the original release note at:
https://numpy.org/devdocs/release/1.20.0-notes.html#deprecations. Did you mean: 'inf'?
```

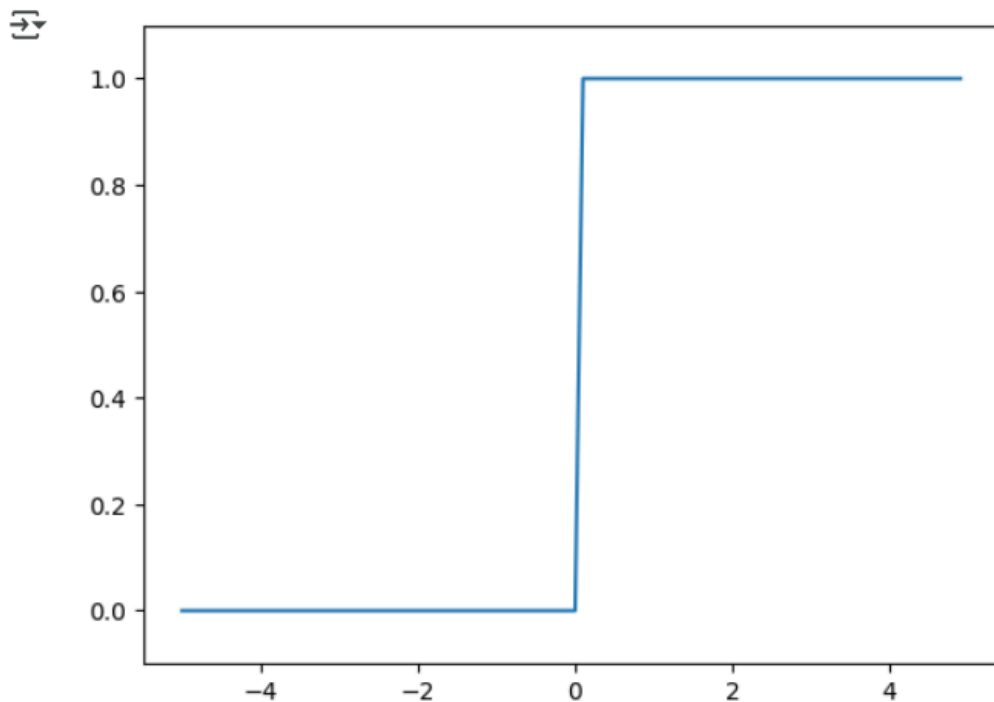
마지막줄 오류

3.2.3 계단함수 그래프

✓
0초 [1] `import numpy as np`
`import matplotlib.pyplot as plt`

[4] `def step_function(x):`
 `return np.array(x > 0, dtype=np.int16) #or int8 처럼 크기 명시`

✓
0초 [5] `x = np.arange(-5.0, 5.0, 0.1)`
`y = step_function(x)`
`plt.plot(x, y)`
`plt.ylim(-0.1, 1.1)`
`plt.show()`



▶ `#step_function 정의할 필요 없이`
`y = np.where(x>0,1,0)`
`# np.where(조건, 조건이 참일때, 조건이 거짓일 때)`

[] 코딩을 시작하거나 AI로 코드를 생성하세요.

3.2.4 시그모이드 함수 구현하기

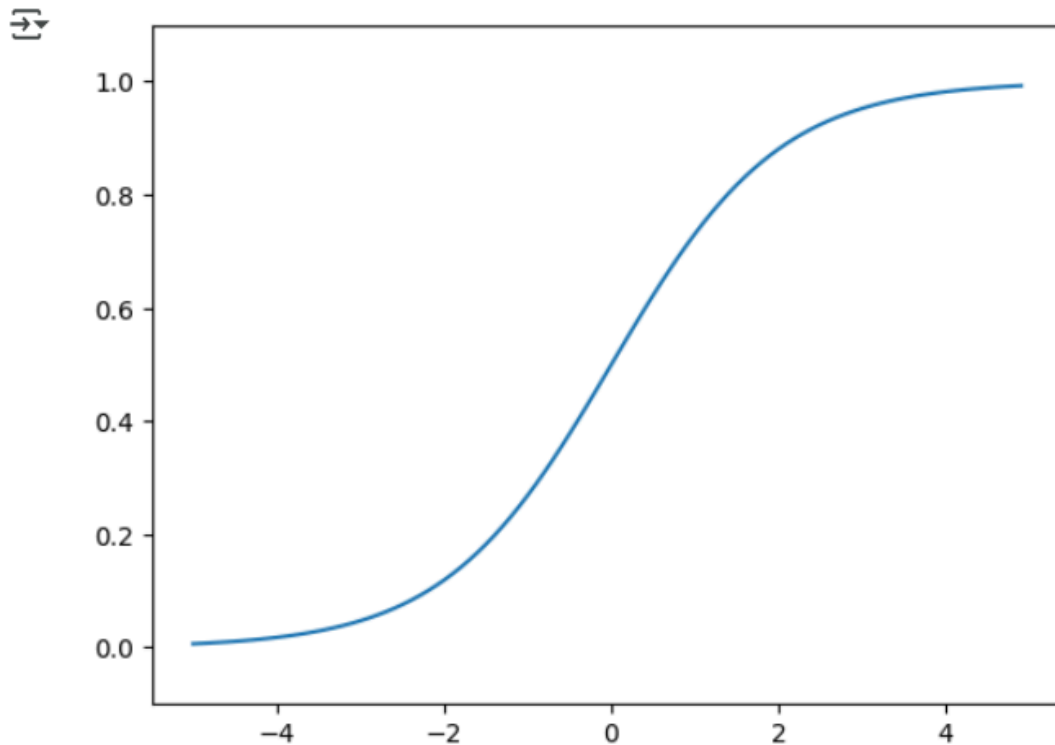
```
[7] #3.2.4 시그모이드 함수 구현하기
def sigmoid(x):
    return 1/(1+np.exp(-x))
```

```
[8] x = np.array([-1.0,1.0,2.0])
```

```
[9] sigmoid(x)
```

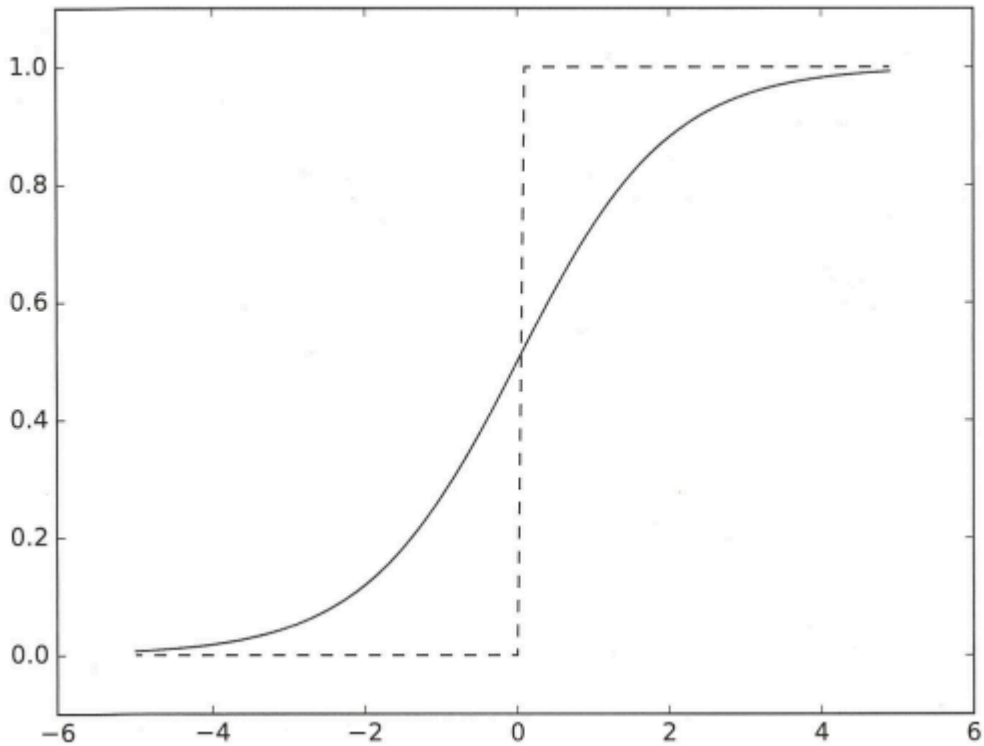
```
↔ array([0.26894142, 0.73105858, 0.88079708])
```

```
[11] #그리기
x = np.arange(-5.0, 5.0, 0.1)
y = sigmoid(x)
plt.plot(x, y)
plt.ylim(-0.1, 1.1)
plt.show()
```



3.2.5 시그모이드 함수 VS 계단 함수

그림 3-8 계단 함수(점선)와 시그모이드 함수(실선)



두가지의 차이점 : “매끄러움”

→ 시그모이드 함수의 매끈함이 아주 중요함

→ 시그모이드 함수에서는 연속적인 실수가 흐른다는 뜻이기 때문

3.2.6 비선형 함수

두가지의 공통점: “비선형 함수”

→ 신경망에서 활성화 함수로는 비선형 함수를 사용해야 한다

→ 여러 층으로 구성하게 되는 은닉층의 이점을 살릴 수 없기 때문

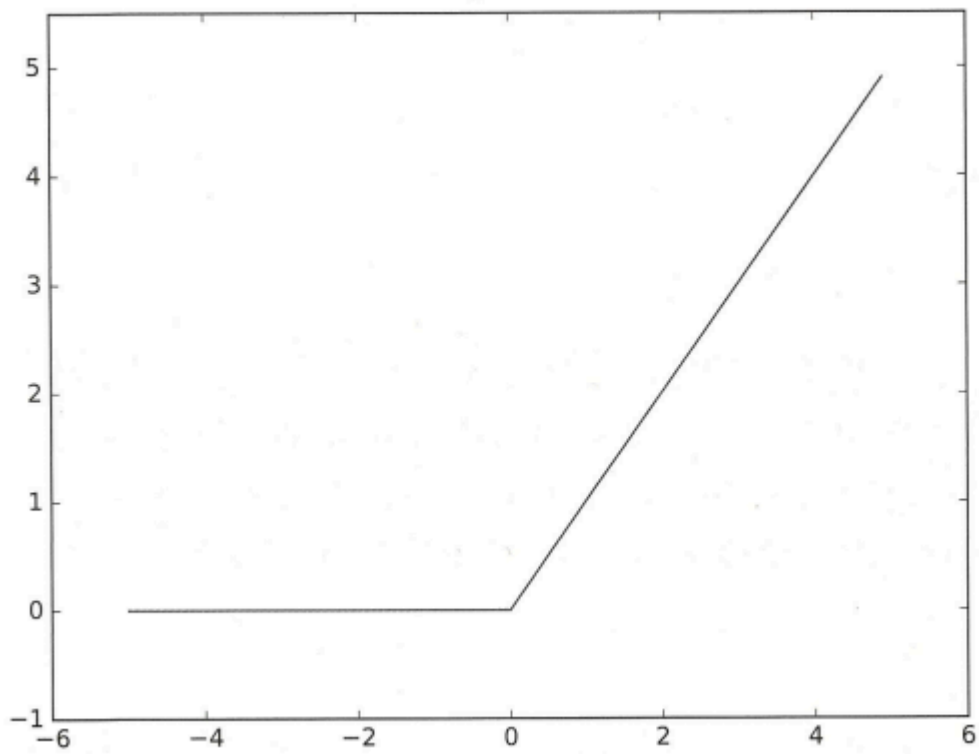
→ 층을 깊게 하는 의미가 없어진다 =?

3.2.7 ReLU 함수

활성화 함수의 종류: 1) 계단 함수 2) 시그모이드 함수 3) ReLU 함수

→ 최근에 주로 이용하는 함수는 ReLU 함수

그림 3-9 ReLU 함수의 그래프*



$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

3.3 다차원 배열 계산

3.3.1. 다차원 배열

A는 1차원 배열이며 원소 4개로 구성되어 있다


```

✓ [1] import numpy as np
0초 A = np.array([1,2,3,4])

✓ [2] print(A)
0초 [1 2 3 4]

✓ [3] np.ndim(A)
0초 1

✓ [4] A.shape
0초 (4,)

✓ [5] A.shape[0]
0초 4

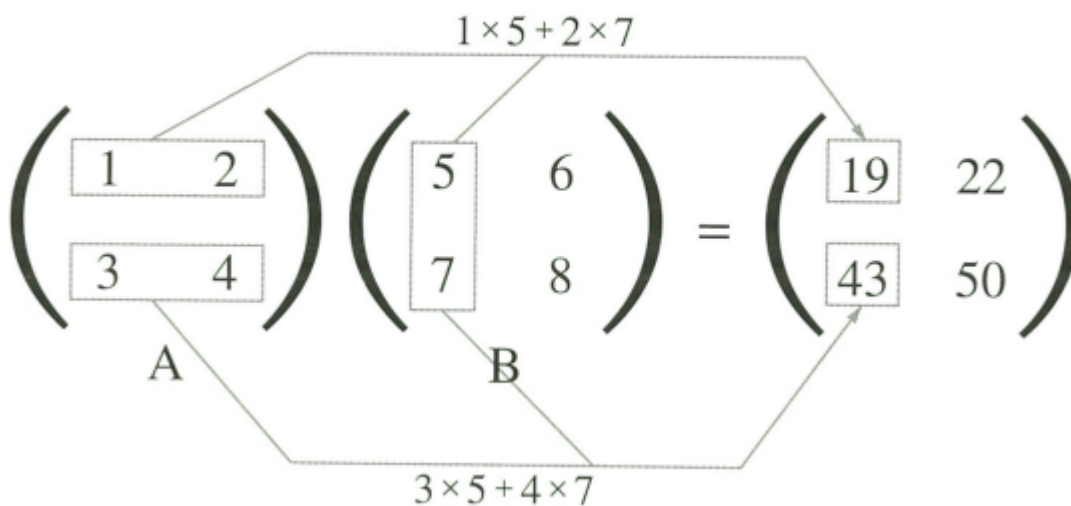
```

ndim ⇒ 배열의 차원 수

shape ⇒ 배열의 형상 (튜플로 반환)

3.3.2. 행렬의 곱

그림 3-11 행렬의 곱 계산 방법



```
A = np.array([[1,2],[3,4]])  
A.shape
```

(2, 2)

```
B = np.array([[5,6],[7,8]])  
B.shape
```

(2, 2)

```
np.dot(A,B)  
# 두 행렬의 곱  
  
array([[19, 22],  
       [43, 50]])
```

일반적인 곱셈과 달리 순서가 바뀌면 답이 다름

주의 *

행렬의 곱셈시 앞 뒤가 달라지면 결과 값도 달라진다.

행렬의 형상을 맞춰주어야 한다.

⇒ 앞선 행렬의 열 수와 뒤의 행 수가 동일해야함

3.3.3. 신경망에서의 행렬 곱

```
[11] X = np.array([1,2])  
X.shape
```

```
⇒ (2,)
```

```
[12] W = np.array([[1,3,5],[2,4,6]])  
print(W)
```

```
⇒ [[1 3 5]  
    [2 4 6]]
```

```
[13] W.shape
```

```
⇒ (2, 3)
```

```
[14] Y = np.dot(X,W)
```

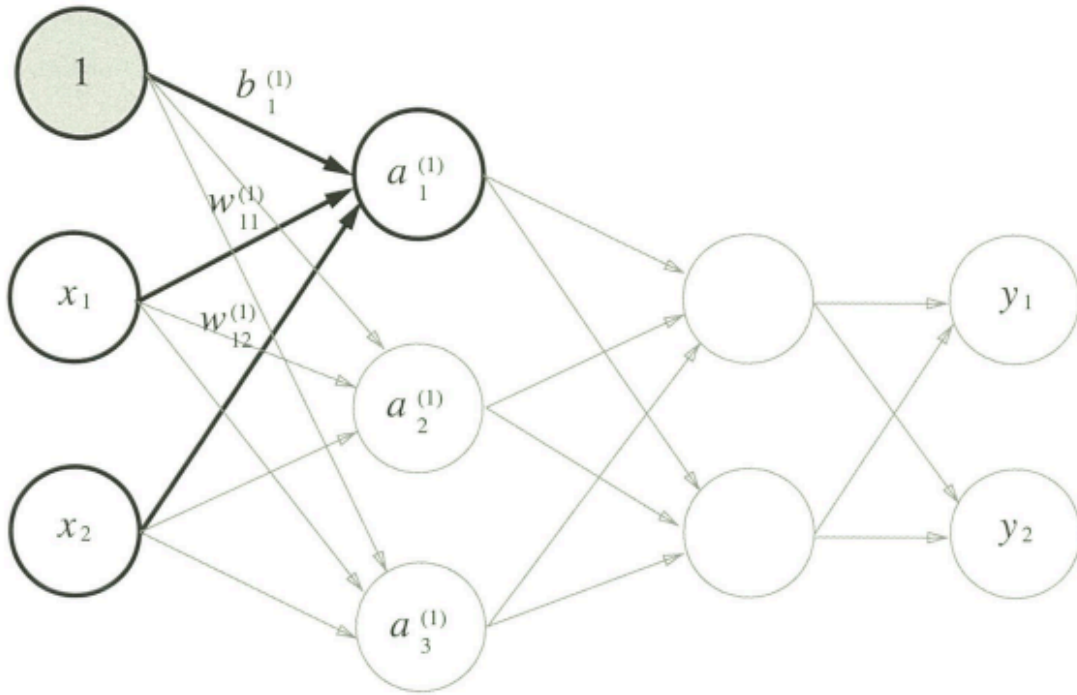
```
[15] print(Y)
```

```
⇒ [ 5 11 17]
```

dot을 이용하여 입력값 x에 가중치를 부여한 결과

3.4. 3층 신경망 구현하기

그림 3-17 입력층에서 1층으로 신호 전달



$$a_1^{(1)} = w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + b_1^{(1)}$$

$$\mathbf{A}^{(1)} = \mathbf{XW}^{(1)} + \mathbf{B}^{(1)}$$

간소화 ver

$$\mathbf{A}^{(1)} = (a_1^{(1)} \ a_2^{(1)} \ a_3^{(1)}), \ \mathbf{X} = (x_1 \ x_2), \ \mathbf{B}^{(1)} = (b_1^{(1)} \ b_2^{(1)} \ b_3^{(1)})$$

$$\mathbf{W}^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \end{pmatrix}$$

```

X = np.array([1.0, 0.5]) 1x2
W1 = np.array([[0.1, 0.3, 0.5], [0.2, 0.4, 0.6]]) 2x3
B1 = np.array([0.1, 0.2, 0.3])

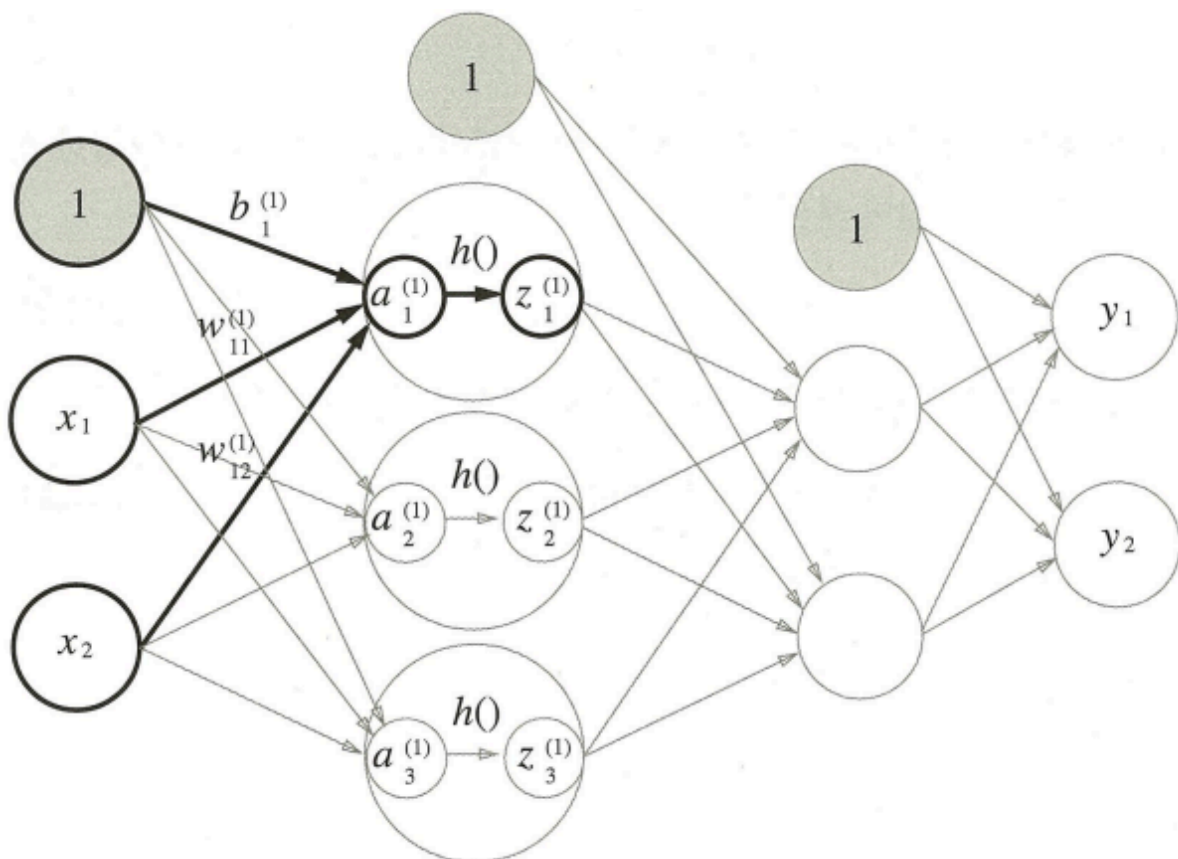
print(W1.shape) # (2, 3)
print(X.shape) # (2,)
print(B1.shape) # (3,)

A1 = np.dot(X, W1) + B1

```

▶ 활성화 함수 적용 (h())

1. 0층 → 1층



```
Z1 = sigmoid(A1)
```

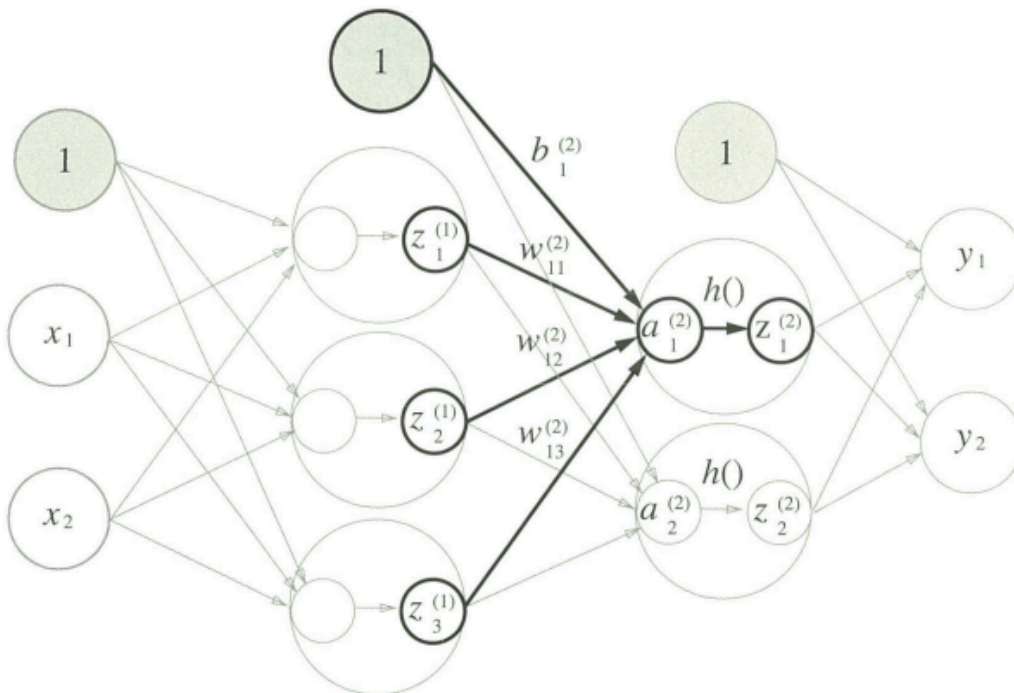
```
print(A1) # [0.3, 0.7, 1.1]
```

```
print(Z1) # [0.57444252, 0.66818777, 0.75026011]
```

⇒ 시그모이드 활성화 함수 적용

2. 1층 → 2층

그림 3-19 1층에서 2층으로의 신호 전달



```
W2 = np.array([[0.1, 0.4], [0.2, 0.5], [0.3, 0.6]])
```

```
B2 = np.array([0.1, 0.2])
```

```
print(Z1.shape) # (3,)
```

```
print(W2.shape) # (3, 2)
```

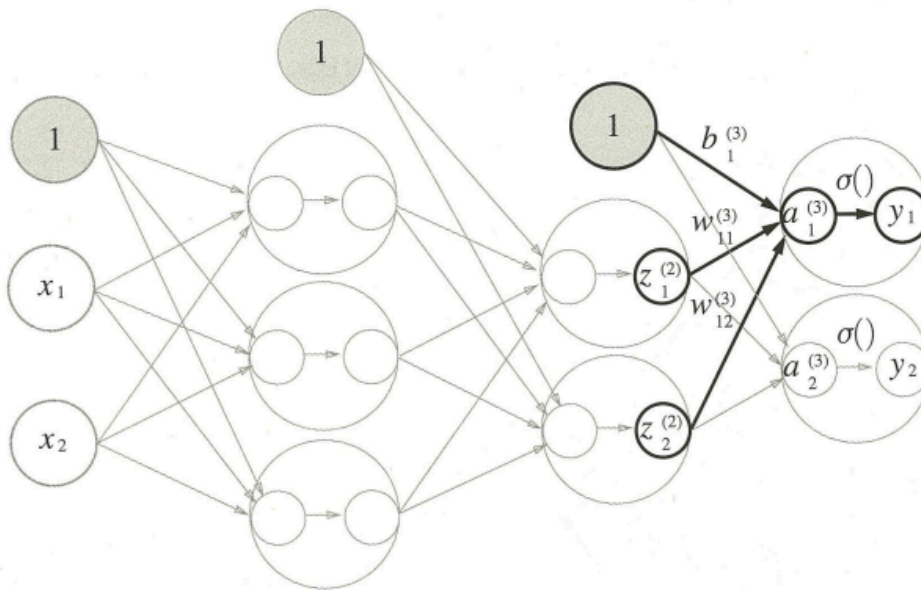
```
print(B2.shape) # (2,)
```

```
A2 = np.dot(Z1, W2) + B2
```

```
Z2 = sigmoid(A2)
```

3. 2층 → 3층(출력층)

그림 3-20 2층에서 출력층으로의 신호 전달



```
def identity_function(x):  
    return x  
  
W3 = np.array([[0.1, 0.3], [0.2, 0.4]])  
B3 = np.array([0.1, 0.2])  
  
A3 = np.dot(Z2, W3) + B3  
Y = identity_function(A3) # 혹은 Y = A3
```

3.4.3 구현 정리

```

def init_network():
    network = {}
    network['W1'] = np.array([[0.1, 0.3, 0.5], [0.2, 0.4, 0.6]])
    network['b1'] = np.array([0.1, 0.2, 0.3])
    network['W2'] = np.array([[0.1, 0.4], [0.2, 0.5], [0.3, 0.6]])
    network['b2'] = np.array([0.1, 0.2])
    network['W3'] = np.array([[0.1, 0.3], [0.2, 0.4]])
    network['b3'] = np.array([0.1, 0.2])

    return network

def forward(network, x):
    W1, W2, W3 = network['W1'], network['W2'], network['W3']
    b1, b2, b3 = network['b1'], network['b2'], network['b3']

    a1 = np.dot(x, W1) + b1
    z1 = sigmoid(a1)
    a2 = np.dot(z1, W2) + b2
    z2 = sigmoid(a2)
    a3 = np.dot(z2, W3) + b3
    y = identity_function(a3)

    return y

network = init_network()
x = np.array([1.0, 0.5])
y = forward(network, x)
print(y) # [ 0.31682708 0.69627909]

```

3.5 출력층 설계하기

분류 VS 회귀(연속적 수치)

분류 : 소프트맥스를 활성화 함수로

회귀 : 항등 함수를 활성화 함수로

3.5.1. 항등 함수와 소프트맥스 함수 구현하기

항등함수 (회귀)

: 입력과 출력이 항상 같다는 뜻의 '항등'

⇒ 입력 신호가 그대로 출력 신호가 간다

소프트 맥스 함수 (분류)

: 지수 함수의 합으로 구성

그림 3-21 항등 함수

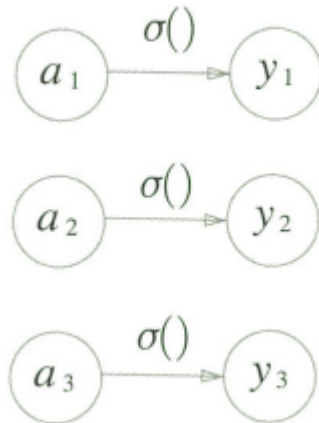
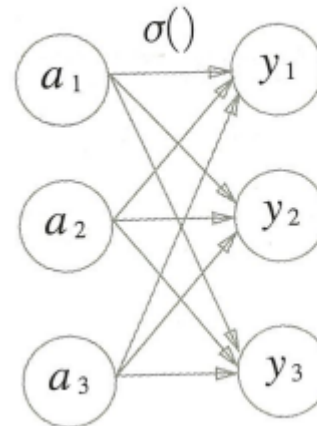


그림 3-22 소프트맥스 함수



```
[19] #소프트맥스 함수 구현
```

```
a = np.array([0.3, 2.9, 4.0])
```

```
exp_a = np.exp(a)
```

```
[20] print(exp_a)
```

```
⇒ [ 1.34985881 18.17414537 54.59815003]
```

```
[21] sum_exp_a = np.sum(exp_a)
```

```
[22] print(sum_exp_a)
```

```
⇒ 74.1221542101633
```

```
[23] y = exp_a / sum_exp_a
```

```
[24] print(y)
```

```
⇒ [0.01821127 0.24519181 0.73659691]
```

3.5.2. 소프트맥스 함수 구현시 주의점

→ 오버플로 문제 발생 가능성 있음

→ 지수 함수를 이용하기 때문에 결과값이 매우 큰 경우가 있고, 큰 값끼리 계산하게 되면 결과 수치가 "불안정"해짐

$$\begin{aligned}
 y_k &= \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)} = \frac{C \exp(a_k)}{C \sum_{i=1}^n \exp(a_i)} \\
 &= \frac{\exp(a_k + \log C)}{\sum_{i=1}^n \exp(a_i + \log C)} \\
 &= \frac{\exp(a_k + C')}{\sum_{i=1}^n \exp(a_i + C')}
 \end{aligned}$$

그래서 log를 이용하여 지수 문제를 해결한다.

⇒ 여기서 말하고자 하는 것은 어떤 값을 더하든 결과가 동일하다는 것

⇒ 오버 플로를 막는 목적으로 보통 최대 값을 이용한다.

⇒ - 최대값

```

>>> a = np.array([1010, 1000, 990])
>>> np.exp(a) / np.sum(np.exp(a)) # 소프트맥스 함수의 계산
array([ nan,  nan,  nan])        # 제대로 계산되지 않는다.
>>>
>>> c = np.max(a)                # c = 1010 (최댓값)
>>> a - c
array([  0, -10, -20])
>>> np.exp(a - c) / np.sum(np.exp(a - c))
array([ 9.99954600e-01,  4.53978686e-05,  2.06106005e-09])

```



3.5.3. 소프트맥스 함수의 특징

1. 0~1.0 사이의 실수로 출력된다.

2. 출력의 총 합은 1이 된다.

⇒ 소프트맥스 함수의 출력을 "확률"로 해석할 수 있다.

** 소프트맥스 함수를 적용해도 원소의 대소관계는 동일하다.

⇒ 신경망을 분류할 땐 출력층의 소프트맥스 함수를 생략해도 되며,

⇒ 일반적으로 생략한다.

So, 신경망을 학습시킬 때만 소프트 맥스 함수를 사용한다.

3.5.4. 출력층의 뉴런 수 정하기

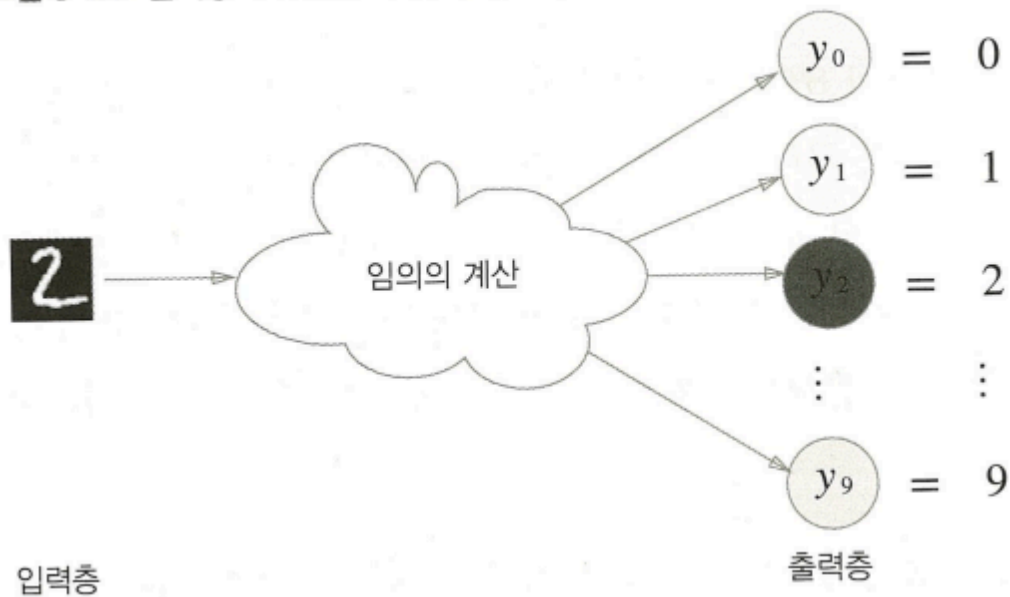
풀어야 하는 문제에 맞추어 정한다.

가령 분류 문제의 경우, 분류하고 싶은 클래스의 갯수만큼 최종 뉴런 수를 정한다.

ex] 0~9까지 숫자로 분류하는 문제

⇒ 출력층의 뉴런 10개로 설정

그림 3-23 출력층의 뉴런은 각 숫자에 대응한다.



3.6. 손글씨 숫자 인식

추론하는 과정(답이 무엇인지)(=순전파)만 구현한다.

3.6.1. MNIST 데이터셋

✓
0초

[4] `import sys, os`
`sys.path.append(os.pardir)`

ⓘ
0초

[5] 제안된 코드에 라이선스가 적용될 수 있습니다. | Adrian123K/dl
`from dataset.mnist import load_mnist`

↩

ModuleNotFoundError

Traceback (most recent call last)

<ipython-input-5-40ce564a4e37> in <cell line: 0>()
----> 1 from dataset.mnist import load_mnist

ModuleNotFoundError: No module named 'dataset'

NOTE: If your import is failing due to a missing package, you can manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the "Open Examples" button below.

OPEN EXAMPLES

다음 단계: 오류 설명

✓
0초

[6] `from PIL import Image`
`import numpy as np`

✓
0초

[7] `def img_show(img):`
 `pil_img = Image.fromarray(np.uint8(img))`
 `pil_img.show()`

ⓘ
0초

[8] 제안된 코드에 라이선스가 적용될 수 있습니다. | BlueWay-KU/Study
`(x_train, t_train), (x_test, t_test) = load_mnist(flatten=True, normalize=False)`

↩

NameError

Traceback (most recent call last)

<ipython-input-8-9411cce90aaa> in <cell line: 0>()
----> 1 (x_train, t_train), (x_test, t_test) = load_mnist(flatten=True, normalize=False)

NameError: name 'load_mnist' is not defined

3. 신경망

21